Course Name: Communication Skills and Technical Writing

Course Code: PC 512

Autumn Semester AY 26-27

Second In Semester Examination, 6th October 2026

Time: 2 Hours **Total Marks: 30**

Marking Scheme: Section 1 = 15 (15 × 1) | Section 2 = 10 (Q.1: 2 × 2.5 = 5; Q.2 = 5) | Section 3 = 5 (10 × 0.5)

Instructions

- The paper consists of three sections.
- Please mention your name and roll number on the answer sheet.
- Please mention the section number and the question number in your answer sheet.

Section 1: Objective Section (15 marks) - 1 mark for each correct answer

Q.1 When is the communication cycle considered complete?

a) When the message is sent by the sender
b) When the sender repeats the message
c) When there is a response from the receiver
d) When the message reaches the receiver

Q.2 The process of communication begins when the sender converts the idea into words or gestures. (Yes / No)

Q.3 Downward communication is best described as flowing:

a) Horizontally between departments
b) Diagonally across unrelated units
c) Only through informal channels
d) From higher to lower levels in the organizational hierarchy

Q.4 Encoding a message is a step in the communication cycle. What does it involve?

a) Using proper language such as words, signs, or objects
b) Sending the message through digital media only
c) The receiver interpreting the message
d) Providing feedback to the sender

Q.5 If the message is successfully encoded and transmitted through a proper channel, can we conclude that the communication process is complete even if the receiver never responds? (Yes / No)

Q.6 Which communication networks are preferred when the speed of communication is important?

a) Y and all-channel networks
b) Wheel and all-channel networks
c) Chain and Y networks
d) Chain and wheel networks

Q.7 The HR department issues an official circular on the notice board announcing a new leave policy. Is this an example of a grapevine in an office? (Yes / No)

Q.8 The Sales manager and the Production manager, who are at the same level, discuss a delivery delay directly and agree on a new schedule without informing the general manager. The problem is solved quickly and the two departments work together better. Later, the general manager is upset that he was not told. Which statement best describes this situation?

a) Horizontal communication, which is wrong because peers should never communicate
b) Horizontal communication, which helped two departments work together but caused conflict as the vertical channel was bypassed
c) Vertical communication caused conflict because two managers at the same level spoke to each other
d) Horizontal communication always builds teamwork and never creates conflict

Q.9 Six sales staff report to one regional head. They never talk to each other, and all information goes through their head. Which network is this?

a) Wheel network
b) Chain
c) All-channel network
d) Y network

Q.10 A company's main server suddenly crashes, and the IT manager must alert all the technical engineers immediately to fix it. Which network should the manager use?

a) Chain or Y network
b) Chain network only
c) Y network only
d) Wheel or all-channel network

Q.11 During a meeting, a manager gives an employee feedback on his mistakes. The employee becomes angry, shouts at the manager and walks out without listening to the rest. Which reason for interpersonal barriers does this show?

a) Cultural variations
b) Emotional outburst
c) Limited vocabulary
d) Communication selectivity

Q.12 A manager emails an employee about a project: the new deadline, the team members, and the required report format. The employee reads only the line about the deadline and ignores the rest. Later, he submits the report in the wrong format. Which reason for interpersonal barriers does this show?

a) Communication selectivity
b) Cultural variations
c) Limited vocabulary
d) Emotional outburst

Q.13 A speaker gives a talk on stock market investing to a group of people with very different education levels, cultures and incomes. He does not find out about their backgrounds or check where they are struggling, and many listeners cannot follow him. Which interpersonal barrier does this show?

a) Incompatibility of verbal and non-verbal messages
b) Differing backgrounds
c) Varied perceptions
d) Blocked categories

Q.14 A head of the Design team asks his assistant to tell the Marketing team that the product launch is on 15 March. The assistant tells a supervisor, who tells a coordinator, who tells Marketing that the launch is on 25 March. The Marketing team, already unhappy with the Design team's work, says the Design team did it on purpose. Both teams now avoid each other. Which **two** organizational barriers does this show?

a) Fear of superiors and information overload
b) Negative tendencies and differing backgrounds
c) Use of inappropriate media and limited vocabulary
d) Too many transfer stations and negative tendencies

Q.15 A manager sends his team 25 emails and a 40-page report every day, thinking more information will keep them better informed. The team feels tired and stops reading them. Is the manager giving importance to the quality of information over the quantity of information? (Yes / No)

Section 2: Email Writing (10 marks)

Q.1 Find the mistakes in the following two emails. (2.5 marks each; total 5 marks)

Email A: Meeting request (Data Scientist)

Subject: Rescheduling Model Review Meeting

Hi Karan,

I'm sorry I haven't written for ages, but I been really busy with the model training. Can we meet at 8 Feb at 14:30 instead? It will be more better for me, as the data pipeline review ends that morning. I hope we can to meet up in the conference room. At the meeting we will discuss the follow points: model accuracy, data pipeline issues and the deployment timeline.

Regards,

Priya Nair

Data Scientist

***Email B:* Unable to help (Cybersecurity Analyst)**

Subject: Re: Request for Penetration Testing Support

Dear Sameer,

Sorry, I don't can help you on this matter, as penetration testing of client applications is outside my team's scope. Please send me the ticket number until Friday at the latest so I can forward it to the right team. I look forward to receiving it so soon as possible. If you require any further informations, please do not hesitate to contact me. I am really appreciate your patience while we sort this out.

Regards,

Arjun Mehta

Cybersecurity Analyst

Q.2 Read the following scenario and write an email based on the instructions. (5 marks)

Scenario: You are a Software Engineer at a technology company, working in a team led by Ms. Kapoor. Your team is releasing a new mobile app update on Monday. On Thursday, during testing, you find a serious bug in the payment module that could cause customers to be charged twice. Fixing and retesting it will take at least two more days, so the release cannot happen on Monday. Use a suitable subject line, greeting and closing, and a formal tone.

Write an email to Ms. Kapoor (about 100 to 120 words) in which you:

- inform her about the bug and explain why it is serious,
- state what you have done so far and how long the fix will take,
- suggest a new release date and request her approval.

Section 3: Comprehension Passage (5 marks) - 0.5 marks for each correct answer

Read the passage carefully and answer the questions that follow. (10 questions × 0.5 mark = 5 marks)

Software as Art

Compared with other artistic media, software has a very short history. Painters and sculptors have worked for millennia, while software has existed for only about 60 years and "software as art" for a few decades. Casting software as an artistic medium may strike many readers as odd, or even objectionable, but a growing body of evidence shows that it is perceived and used in exactly this way.

Programmers passionate about their work often sense an aesthetic underlying their approach. They know what is pleasant to perform and what is unpleasant to endure, what is beautiful to behold and what is intolerably ugly. This raises questions. What are the bases for these judgments? Do they extend beyond engineering? Can software transcend its role as a utilitarian artifact and become art? Only recently have these questions been explored in earnest, and the participants are not software practitioners but artists rooted in the new-media community. New-media festivals now give awards for "artistic software," and museums have exhibited software. Ironically, the programming community, who best understand the medium, have largely overlooked this first period of software art.

The programmer's aesthetic

Donald Knuth has written about the art inherent in programming since the 1970s. As the sole developer of large projects such as the TeX typesetting program, and as a Stanford professor who has judged many students' programs, he has a refined sense of software aesthetics. He chose "Computer Programming as an Art" as the topic of his 1974 Turing Award lecture. There he argued that both the act of programming and the finished program have an aesthetic. A programmer's style can gratify the programmer and possibly the reader. He described the thrill of reading assembly code by an accomplished programmer and declared that beauty can be found in programs. In a later lecture series he compared beautiful programs to literature or music, whose tasks are elegantly stated and whose parts come together symphonically. He has also found some programs painful to read, and others witty.

Donald Knuth named the properties that give a program's readers pleasure or pain: correctness, maintainability, readability, lucidity, grace of interaction with users, and efficiency. Practising developers largely agree. Discussions of "beautiful software" on the Slashdot website name the same qualities, so the developer's aesthetic has barely changed in thirty years.

Knuth later developed "literate programming," which aims to make writing and reading software more enjoyable. It turns the usual approach on its head. Instead of a programmer documenting the code, the documentarian embeds code in a narrative document, so the program can be written as a story, free from the structural constraints of the programming language. Two tools support this: "tangle" extracts compilable code from the finished story, and "weave" produces a typeset version. The approach improves maintainability, readability and lucidity. The best-known examples are Knuth's own TeX and Metafont programs, published as *The TeXbook* and *The Metafont Book*, which many regard as works of art by an accomplished artist.

Lucidity is often pursued as an aesthetic goal in itself and can produce the most powerful reaction of all. A lucid program resembles a profound mathematical theorem or physical theory in its beauty, and mathematically minded computer scientists usually produce the finest examples. Dijkstra's short algorithm for serializing access to shared data among concurrent processes solves an important problem. It also shows its author's deep understanding of concurrency, and a reader who follows it gains that understanding too. Its primary purpose is practical, but the way it achieves that purpose transcends the practical, much as a work of art does. Languages based on mathematical calculi, such as ML, Scheme, Prolog and Occam, encourage this kind of lucidity. A Prolog definition of insertion sort is both a precise mathematical description of a sorted list and a program that actually sorts one when executed.

Art for art's sake

Knuth also described a less pragmatic aesthetic, which he called "art for art's sake." It is the pleasure programmers and readers take in programs that operate under artificial constraints. The art world is full of works appreciated for the challenge the artist faced and how well it was met, such as Michelangelo's Sistine Chapel or Christo and Jeanne-Claude's The Gates in Central Park. Software has its equivalents:

- Quines: a term coined by Douglas Hofstadter for programs that output themselves. They look simple but are maddeningly self-referential.
- Obfuscated programs: the annual Obfuscated C contest scores entries on brevity and on how hard it is for readers to predict what the program will do.
- Polyglots: programs that are legal in more than one programming language, often using comments to hide constructs that would otherwise be illegal. One 1991 example satisfies seven compilers.

What these programs share is that they are valued not for utility, which is usually negligible, but for their authors' technical mastery and wit. This goes against the traditional idea of software, and that is where the art lies. Yet only people close to the field can truly appreciate them.

The software artist

Must software art remain closed to the public? Sharon Hopkins's poetry shows it need not. Her poem "listen," a syntactically valid Perl program, was published in The Economist in 1995. It uses a programming language to convey human sentiment to humans rather than instructions to a computer, and it casts the language in a strange, unfamiliar light. A programming language is a highly constrained language, which makes it a natural medium for a poet at home with the sonnet or haiku.

Hopkins introduces the software artist, who differs from the programmer-artists above because they are first and foremost an artist, and the medium serves their self-expression. She respects Perl's syntax but ignores its semantics, using English meaning instead. This might horrify programmers, whose programs always function in some way, even if absurdly. But it lets her reach a large audience, and it carries a radical message to programmers: software can be harnessed for art, and violence may be done to it in the process.

Doing violence to a medium has a long history in art. Impressionism and abstraction once disoriented and angered painting audiences, and software art is developing in a similar way. It began with playful experiments such as quines and obfuscated programs, which removed software from its utilitarian context and highlighted its expressive possibilities.

Software art and the concept of software

Today software is pervasive and hidden inside countless devices, controlling how they behave and how people use them. It is logical but frequently bug-ridden, and it is open to infiltration and infection. Because software is so familiar, artists are drawn to it and, as in much contemporary art, they make works about the "concept" of the medium itself. Prize winners at recent festivals show what they explore:

Transmediale (Berlin) was the first festival to offer a software art prize, in 2001. Its jury demanded programs that went beyond the purely functional. Adrian Ward's "Auto-Illustrator" looks like an ordinary graphics program, but its tools resist the user and seem to act at their own will. In 2002, Alex McLean's "forkbomb.pl" copied itself repeatedly until it brought the computer to its knees, and "TraceNoizer" built a permanent home page for a person out of unrelated snippets of the Web. READ_ME (Moscow, 2002) was devoted solely to software art. Its winners included "DeskSwap," a screen saver that reveals other users' desktops, and "Textension," which animates typed text.

This work addresses, and often confronts, the popular concept of software and its role in our lives. Unlike Knuth's aesthetic, which mainly developers appreciate, the general public can engage with it. Programmer-artists may feel their legacy has been overlooked by the art establishment, but their work does require technical knowledge to appreciate. As software becomes more ubiquitous, audiences will grow more sophisticated, just as they eventually came to appreciate Stravinsky and Joyce.

A rapidly evolving medium

Unlike most media, software evolves very quickly. From raw binary code 60 years ago, it has grown into a broad palette of programming languages, code processors and programs, from word processors to viruses. Few media offer so many ways to express oneself. In the thirty years since Knuth first addressed the art of programming, the field has been active but static in scope. With artists only beginning to explore the medium, programmer-artists now have a chance to extend its boundaries, drawing on the virtuosity that comes from an intimate relationship with their medium.

Questions

Q.1 Who does the author say has mainly been exploring whether software can be art?

a) Professional software engineers
b) Hardware manufacturers
c) Computer science students
d) Artists rooted in the new-media community

Q.2 What did Knuth argue in his 1974 Turing Award lecture?

a) That programming is purely a mathematical activity
b) That both the act of programming and the finished program have an aesthetic
c) That software should never be compared to art
d) That programs should be as short as possible

Q.3 Which of these is NOT among the properties Knuth said give readers pleasure or pain?

a) Maintainability
b) Lucidity
c) Commercial popularity
d) Grace of interaction with users

Q.4 What do the tools 'tangle' and 'weave' do in literate programming?

a) Tangle extracts compilable code from the story and weave produces a typeset version
b) Tangle translates code between languages and weave compresses it
c) Tangle deletes comments and weave adds them
d) Tangle debugs code and weave optimizes it

Q.5 Why does the author say Dijkstra's concurrency algorithm has artistic qualities?

a) It is visually striking
b) It was the first concurrent program
c) It solves a real problem while revealing deep understanding and giving the reader insight
d) It was written to be deliberately obscure

Q.6 What do quines, obfuscated programs and polyglots have in common?

a) They are widely used in commercial software
b) They were all created by Knuth
c) They are easily appreciated by the general public
d) They are valued for technical mastery and wit rather than utility

Q.7 How does Sharon Hopkins's Perl poetry work?

a) It is written in English and translated to Perl by a compiler
b) It follows Perl's syntax but uses English semantics
c) It is a functional program that sorts words

d) It ignores Perl's syntax but follows its semantics

Q.8 Why does the author mention impressionism and abstraction?

a) To show that painters also use programming
b) To explain how festival winners were chosen
c) To show that doing 'violence' to a medium for art has precedent and can initially disorient audiences
d) To argue that painting is superior to software art

Q.9 What did Alex McLean's 'forkbomb.pl' do?

a) It displayed other users' desktops as a screen saver
b) It animated typed text
c) It built a permanent home page from unrelated Web snippets
d) It repeatedly copied itself until it overwhelmed the computer

Q.10 What opportunity do programmer-artists now have, according to the closing paragraph?

a) To win the Turing Award
b) To leave programming for painting
c) To extend the field's boundaries using their intimate knowledge of the medium
d) To replace software artists in galleries